

Introduction of Javascript

What is JavaScript?

JavaScript was initially created to “make web pages alive”.

The programs in this language are called scripts. They can be written right in a web page’s HTML and run automatically as the page loads.

Scripts are provided and executed as plain text. They don’t need special preparation or compilation to run.

Why is it called JavaScript?

When JavaScript was created, it initially had another name: “LiveScript”. But Java was very popular at that time, so it was decided that positioning a new language as a “younger brother” of Java would help.

But as it evolved, JavaScript became a fully independent language with its own specification called ECMAScript, and now it has no relation to Java at all.

JavaScript can execute not only in the browser, but also on the server, or actually on any device that has a special program called the JavaScript engine.

What can in-browser JavaScript do?

For instance, in-browser JavaScript is able to:

- Add new HTML to the page, change the existing content, and modify styles.
- React to user actions, run on mouse clicks, pointer movements, and key presses.
- Send requests over the network to remote servers, download and upload files (so-called [AJAX](#) and [COMET](#) technologies).
- Get and set cookies, ask questions to the visitor, show messages.
- Remember the data on the client-side (“local storage”).

What makes JavaScript unique?

- Full integration with HTML/CSS.
- Simple things are done simply.
- Supported by all major browsers and enabled by default.

Code editors

1. IDE

The term IDE (Integrated Development Environment) refers to a powerful editor with many features that usually operates on a “whole project.” As the name suggests, it’s not just an editor, but a full-scale “development environment.”

If you haven’t selected an IDE yet, consider the following options:

Visual Studio Code (cross-platform, free).

WebStorm (cross-platform, paid).

2. Lightweight editors

“Lightweight editors” are not as powerful as IDEs, but they’re fast, elegant and simple.

They are mainly used to open and edit a file instantly.

The main difference between a “lightweight editor” and an “IDE” is that an IDE works on a project-level, so it loads much more data on start, analyzes the project structure if needed and so on. A lightweight editor is much faster if we need only one file.

There are many options, for instance:

Sublime Text (cross-platform, shareware).

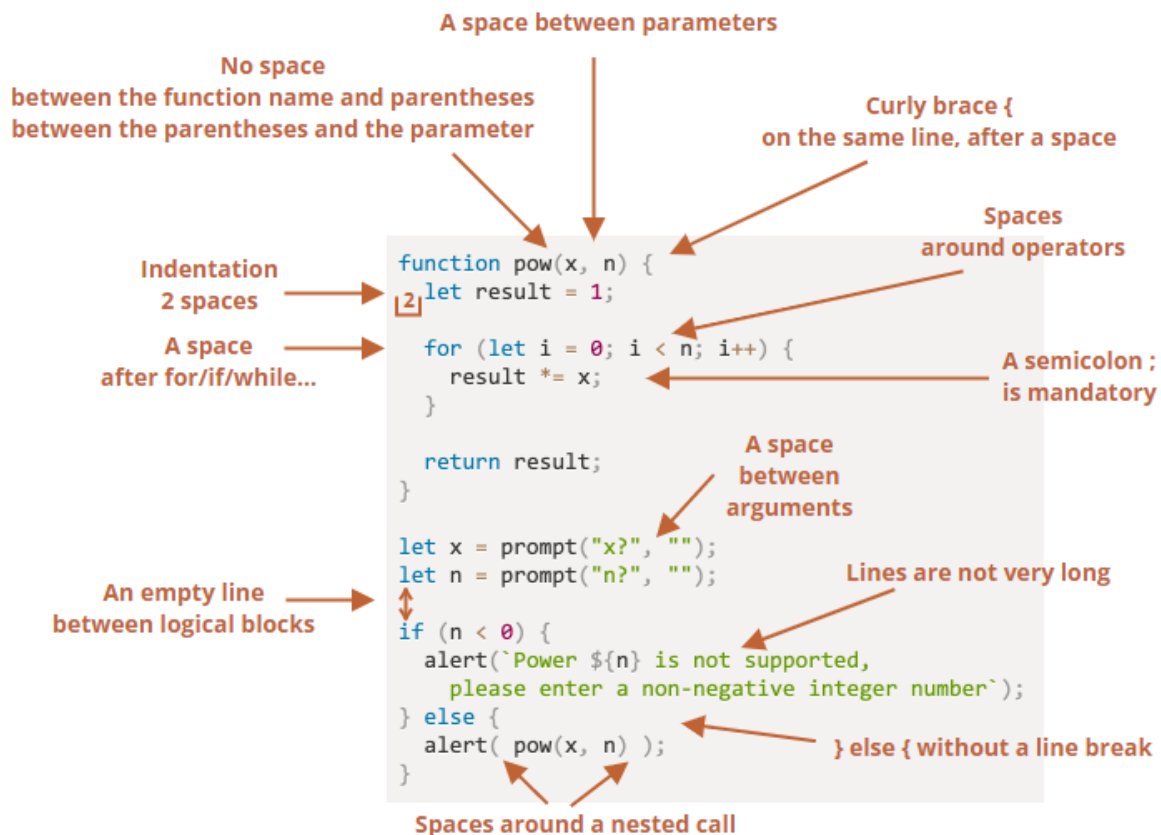
Notepad++ (Windows, free).

Vim and **Emacs** are also cool if you know how to use them.

Coding Style:

Our code must be as clean and easy to read as possible.

That is actually the art of programming – to take a complex task and code it in a way that is both correct and human-readable. A good code style greatly assists in that.



Curly Braces

In most JavaScript projects curly braces are written in “Egyptian” style with the opening brace on the same line as the corresponding keyword – not on a new line. There should also be a space before the opening bracket, like this:

```
if (condition) {  
  // do this  
  // ...and that  
  // ...and that  
}
```

The “script” tag

JavaScript programs can be inserted almost anywhere into an HTML document using the `<script>` tag.

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Before the script...</p>
  <script>
    alert( 'Hello, world!' );
  </script>
  <p>...After the script.</p>
</body>
</html>
```

External scripts

If we have a lot of JavaScript code, we can put it into a separate file.

Script files are attached to HTML with the `src` attribute:

```
<script src="/path/to/script.js"></script>
```

Variables

Most of the time, a JavaScript application needs to work with information. Here are two examples:

- An online shop – the information might include goods being sold and a shopping cart.
- A chat application – the information might include users, messages, and much more.

Variables are used to store this information.

A variable:

The statement below creates (in other words: *declares*) a variable with the name “message”:

```
let message;
```

Now, we can put some data into it by using the assignment operator =:

```
let message;  
message = 'Hello'; // store the string 'Hello' in the variable named message
```

A real-life analogy

We can easily grasp the concept of a “variable” if we imagine it as a “box” for data, with a uniquely-named sticker on it.

For instance, the variable message can be imagined as a box labeled "message" with the value "Hello!" in it:

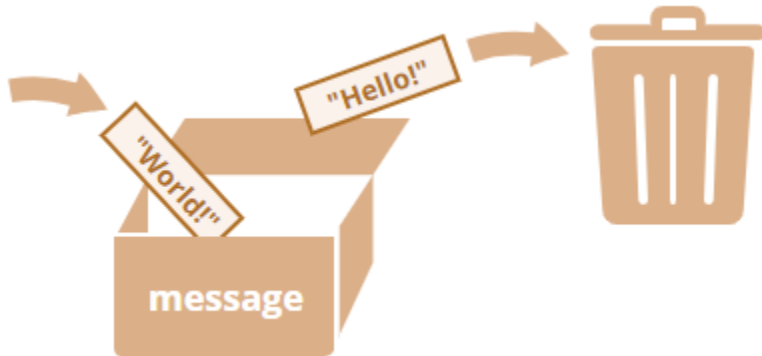


We can put any value in the box.

We can also change it as many times as we want:

```
let message;  
message = 'Hello!';  
message = 'World!'; // value changed  
alert(message);
```

When the value is changed, the old data is removed from the variable:



Variable naming:

There are two limitations on variable names in JavaScript:

- The name must contain only letters, digits, or the symbols \$ and _.
- The first character must not be a digit.

Examples of valid names:

```
let userName;  
let test123;
```

Constants

To declare a constant (unchanging) variable, use `const` instead of `let`

Ex:

```
const name = "creative";
```

- **let** – is a modern variable declaration.
- **var** – is an old-school variable declaration. Normally we don't use it at all, but we'll cover subtle differences from `let` in the chapter The old "var", just in case you need them.
- **const** – is like `let`, but the value of the variable can't be changed.

Interaction: alert, prompt, confirm

- **alert**

It shows a message and waits for the user to press “OK”.

Ex:

```
alert("Hello");
```

- **prompt**

The function prompt accepts two arguments:

Title: The text to show the visitor.

Default: An optional second parameter, the initial value for the input field.

Example :

```
result = prompt(title, [default]);
```

- **confirm**

Example:

```
result = confirm(question);
```

Type Conversions

- **String Conversion :**

String conversion happens when we need the string form of a value.

For example, alert(value) does it to show the value.

We can also call the String(value) function to convert a value to a string:

```
let value = true;  
alert(typeof value); // boolean  
value = String(value); // now value is  
a string "true"  
alert(typeof value); // string
```

- **Numeric Conversion**

Numeric conversion in mathematical functions and expressions happens automatically.

```
let str = "123";  
alert(typeof str); // string  
let num = Number(str); //  
becomes a number 123  
alert(typeof num); // number
```

- **Boolean Conversion**

Boolean conversion is the simplest one.

It happens in logical operations (later we'll meet condition tests and other similar things) but can also be performed explicitly with a call to `Boolean(value)`.

The conversion rule:

Values that are intuitively “empty”, like 0, an empty string, null, undefined, and NaN, become false.

Other values become true

```
alert( Boolean(1) ); // true  
alert( Boolean(0) ); // false  
alert( Boolean("hello") ); // true  
alert( Boolean("") ); // false
```


Conditional branching

- **The “if” statement**

The if(...) statement evaluates a condition in parentheses and, if the result is true, executes a block of code.

Example:

```
if (year == 2015) {  
  alert( "That's correct!" );  
  alert( "You're so smart!" );  
}
```

- **The “else” clause**

The if statement may contain an optional else block. It executes when the condition is falsy.

Example:

```
if (year == 2015) {  
  alert ( 'You guessed it right!' );  
} else {  
  alert ( 'How can you be so wrong?' ); // any value except 2015  
}
```

- **Conditional operator ‘?’**

Sometimes, we need to assign a variable depending on a condition.

The so-called “conditional” or “question mark” operator lets us do that in a shorter and simpler way.

CREATIVE DESIGN & MULTIMEDIA INSTITUTE

The operator is represented by a question mark ?. Sometimes it's called "ternary", because the operator has three operands. It is actually the one and only operator in JavaScript which has that many.

Syntax:

```
let result = condition? value1: value2;
```

Example:

```
let accessAllowed = (age > 18) ? true: false;
```

- **Nullish coalescing operator '??'**

The nullish coalescing operator is written as two question marks??.

As it treats null and undefined similarly, we'll use a special term here, in this article. For brevity, we'll say that a value is "defined" when it's neither null nor undefined

The result of a ?? b is:

- if a is defined, then a,
- if a isn't defined, then b.

```
let user;  
alert(user ?? "Anonymous"); // Anonymous (user is undefined)
```

- **Loops: while and do while and for**

1. While Loop:

The while loop has the following syntax:

Syntax:

```
while (condition) {  
    // code  
    // so-called "loop body"  
}
```

Example:

```
let i = 0;  
while (i < 3) { // shows 0, then 1,  
    then 2  
    alert ( i );  
    i++;  
}
```

2. Do While loop :

The condition check can be moved below the loop body using the do..while syntax:

Syntax:

```
do {  
    // loop body  
} while (condition);
```

Example:

```
let i = 0;  
do {  
    alert( i );  
    i++;  
} while (i < 3);
```

3. For Loop :

The for loop is more complex, but it's also the most commonly used loop.

Syntax:

```
for (begin; condition; step) {  
    // ... loop body...  
}
```

Example:

```
for (let i = 0; i < 3; i++) { // shows 0,  
    then 1, then 2  
    alert(i);  
}
```

The "switch" statement

A switch statement can replace multiple if checks.

It gives a more descriptive way to compare a value with multiple variants.

Syntax:

```
switch(x) {  
    case 'value1': // if (x === 'value1')  
        ...  
        [break]  
    case 'value2': // if (x === 'value2')  
        ...  
        [break]  
    default:  
        ...  
        [break]  
}
```

Example:

```
let a = 2 + 2;
switch (a) {
  case 3:
    alert( 'Too small' );
    break;
  case 4:
    alert( 'Exactly!' );
    break;
  case 5:
    alert( 'Too big' );
    break;
  default:
    alert( "I don't know such values" );
}
```

Functions:

Functions are the main “building blocks” of the program. They allow the code to be called many times without repetition.

We’ve already seen examples of built-in functions, like `alert(message)`, `prompt(message, default)` and `confirm(question)`. But we can create functions of our own as well.

Function Declaration:

```
function showMessage() {
  alert( 'Hello everyone!' );
}
```

CREATIVE DESIGN & MULTIMEDIA INSTITUTE

The function keyword goes first, then goes the name of the function, then a list of parameters between the parentheses (comma-separated, empty in the example above, we'll see examples later) and finally the code of the function, also named "the function body", between curly braces.

```
function name (parameter1, parameter2, ... parameterN) {  
  // body  
}
```

Our new function can be called by its name: showMessage ()

```
function showMessage() {  
  alert( 'Hello everyone!' );  
}  
showMessage();  
showMessage();
```

Local variables:

A variable declared inside a function is only visible inside that function.

Example:

```
function showMessage() {  
  let message = "Hello, I'm JavaScript!"; // local variable  
  
  alert( message );  
}  
  
showMessage(); // Hello, I'm JavaScript!  
  
alert( message ); // <-- Error! The variable is local to the function
```

Outer variables:

A function can access an outer variable as well, for example:

```
let userName = 'John';
function showMessage() {
  let message = 'Hello, ' + userName;
  alert(message);
}
showMessage(); // Hello, John
```

The function has full access to the outer variable. It can modify it as well.

```
let userName = 'John';
function showMessage() {
  userName = "Bob"; // (1) changed the outer variable
  let message = 'Hello, ' + userName;
  alert(message);
}
alert( userName ); // John before the function call
showMessage();
alert( userName ); // Bob, the value was modified by the function
```

Parameters:

We can pass arbitrary data to functions using parameters.

In the example below, the function has two parameters: from and text.

```
function showMessage(from, text) { // parameters: from, text
  alert(from + ': ' + text);
}
showMessage('Ann', 'Hello!'); // Ann: Hello! (*)
showMessage('Ann', "What's up?"); // Ann: What's up? (**)
```

When the function is called in lines (*) and (**), the given values are copied to local variables `from` and `text`. Then the function uses them.

Default values

If a function is called, but an argument is not provided, then the corresponding value becomes undefined.

For instance, the aforementioned function **`showMessage(from, text)`** can be called with a single argument:

```
showMessage("Ann");
```

That's not an error. Such a call would output `"*Ann*: undefined"`. As the value for `text` isn't passed, it becomes undefined.

We can specify the so-called “default” (to use if omitted) value for a parameter in the function declaration, using `=`:

```
function showMessage(from, text = "no text given") {
  alert( from + ": " + text );
}
showMessage("Ann"); // Ann: no text given
```


Alternative default parameters:

Sometimes it makes sense to assign default values for parameters at a later stage after the function declaration.

We can check if the parameter is passed during the function execution, by comparing it with `undefined`:

```
function showMessage(text) {  
  // ...  
  if (text === undefined) { // if the parameter is missing  
    text = 'empty message';  
  }  
  alert(text);  
}  
showMessage(); // empty message
```

...Or we could use the `||` operator:

```
function showMessage(text) {  
  // if text is undefined or otherwise false, set it to  
  'empty'  
  text = text || 'empty';  
  ...  
}
```

Returning a value:

A function can return a value back into the calling code as the result.

The simplest example would be a function that sums two values:

Example:

```
function sum(a, b) {  
  return a + b;  
}  
  
let result = sum(1, 2);  
alert (result ); // 3
```

Naming a function:

Functions are actions. So their name is usually a verb. It should be brief, as accurate as possible and describe what the function does, so that someone reading the code gets an indication of what the function does.

It is a widespread practice to start a function with a verbal prefix which vaguely describes the action. There must be an agreement within the team on the meaning of the prefixes.

For instance, functions that start with "show" usually show something.

Function starting with...

- "get..." – return a value,
- "calc..." – calculate something,
- "create..." – create something,
- "check..." – check something and return a boolean, etc.

Callback functions

Let's look at more examples of passing functions as values and using function expressions.

We'll write a function `ask(question, yes, no)` with three parameters:

question

Text of the question

yes

Function to run if the answer is "Yes"

no

Function to run if the answer is "No"

Example:

```
function ask(question, yes, no) {  
  if (confirm(question)) yes()  
  else no();  
}  
function showOk() {  
  alert( "You agreed." );  
}  
function showCancel() {  
  alert( "You canceled the execution." );  
}  
// usage: functions showOk, showCancel are passed as arguments to ask  
ask("Do you agree?", showOk, showCancel);
```

Function Expression vs Function Declaration:

Function Declaration: a function, declared as a separate statement, in the main code flow:

```
// Function Declaration
function sum(a, b) {
  return a + b;
}
```

Function Expression: a function, created inside an expression or inside another syntax construct. Here, the function is created on the right side of the “assignment expression” =:

```
// Function Expression
let sum = function(a, b) {
  return a + b;
};
```

Arrow functions

There's another very simple and concise syntax for creating functions, that's often better than Function Expressions.

It's called “arrow functions”, because it looks like this:

Example:

```
let func = (arg1, arg2, ..., argN) => expression;
```

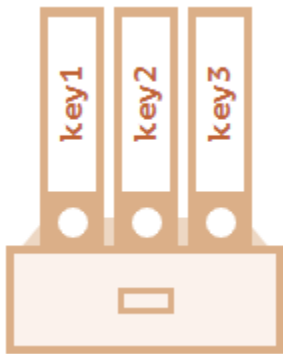
Objects:

As we know from the chapter Data types, there are eight data types in JavaScript. Seven of them are called “primitive”, because their values contain only a single thing (be it a string or a number or whatever).

In contrast, objects are used to store keyed collections of various data and more complex entities. In JavaScript, objects penetrate almost every aspect of the language. So we must understand them first before going in-depth anywhere else.

An object can be created with figure brackets {...} with an optional list of properties. A property is a “key: value” pair, where key is a string (also called a “property name”), and value can be anything.

We can imagine an object as a cabinet with signed files. Every piece of data is stored in its file by the key. It's easy to find a file by its name or add/remove a file.



An empty object (“empty cabinet”) can be created using one of two syntaxes:



Literals and properties

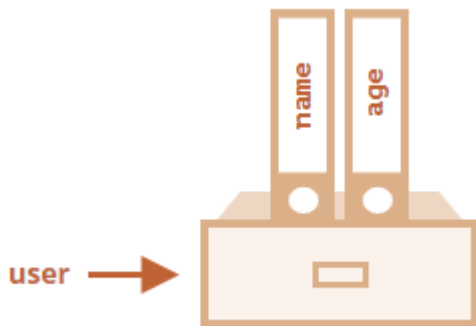
We can immediately put some properties into {...} as “key: value” pairs:

```
let user = { // an object
  name: "John", // by key "name" store
  value "John"
  age: 30 // by key "age" store value 30
};
```

A property has a key (also known as “name” or “identifier”) before the colon ":" and a value to the right of it.

In the user object, there are two properties:

1. The first property has the name "name" and the value "John".
2. The second one has the name "age" and the value 30.



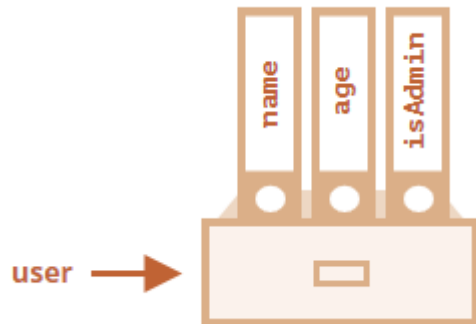
We can add, remove and read files from it at any time.

Property values are accessible using the dot notation:

```
// get property values of the object:
alert( user.name ); // John
alert( user.age ); // 30
```

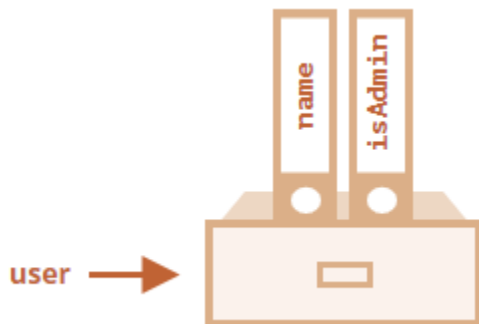
The value can be of any type. Let's add a Boolean one:

```
user.isAdmin = true;
```



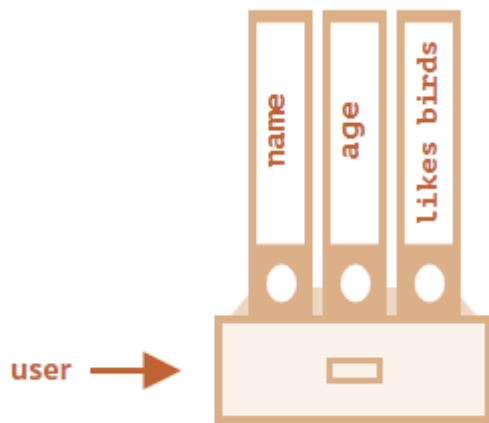
To remove a property, we can use the delete operator:

```
delete user.age;
```



We can also use multiword property names, but then they must be quoted:

```
let user = {  
  name: "John",  
  age: 30,  
  "likes birds": true // multiword property  
                        name must be quoted  
};
```



The "for.. in" loop

To walk over all keys of an object, there exists a special form of the loop: for..in. This is a completely different thing from the for(;;) construct that we studied before.

Syntax:

```
for (key in object) {  
  // executes the body for each key among object properties  
}
```

Example:

```
let user = {  
  name: "John",  
  age: 30,  
  isAdmin: true  
};  
for (let key in user) {  
  // keys  
  alert( key ); // name, age, isAdmin  
  // values for the keys  
  alert( user[key] ); // John, 30, true  
}
```


CREATIVE DESIGN & MULTIMEDIA INSTITUTE

One of the fundamental differences of objects versus primitives is that objects are stored and copied “by reference”, whereas primitive values: strings, numbers, Booleans, etc – are always copied “as a whole value”.

That’s easy to understand if we look a bit under the hood of what happens when we copy a value.

Let’s start with a primitive, such as a string.

Here we put a copy of message into phrase:

```
let message = "Hello!";  
let phrase = message;
```



A variable assigned to an object stores not the object itself, but its “address in memory” – in other words “a reference” to it.

```
let user = {  
  name: "John"  
};
```

When an object variable is copied, the reference is copied, but the object itself is not duplicated.

```
let fruits = ["Apple", "Orange", "Plum"];  
alert( fruits[0] ); // Apple  
alert( fruits[1] ); // Orange  
alert( fruits[2] ); // Plum
```

Array:

An array is a special variable, which can hold more than one value at a time.

Declaration:

```
let arr = new Array();  
let arr = [];
```

Almost all the time, the second syntax is used. We can supply initial elements in the brackets:

```
let fruits = ["Apple", "Orange", "Plum"];
```

Array elements are numbered, starting with zero.

We can get an element by its number in square brackets:

Methods push/pop, shift/unshift

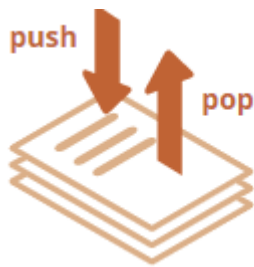
- A queue is one of the most common uses of an array. In computer science, this means an ordered collection of elements which supports two operations:
- Push appends an element to the end.
- Shift get an element from the beginning, advancing the queue, so that the 2nd element becomes the 1st.



- Arrays support both operations.

CREATIVE DESIGN & MULTIMEDIA INSTITUTE

- In practice we need it very often. For example, a queue of messages that need to be shown on-screen.
- There's another use case for arrays – the data structure named stack.
- It supports two operations:
- Push adds an element to the end.
- Pop takes an element from the end.
- So new elements are added or taken always from the “end”.
- A stack is usually illustrated as a pack of cards: new cards are added to the top or taken from the top:



- For stacks, the latest pushed item is received first, that's also called LIFO (Last-In-First-Out) principle. For queues, we have FIFO (First-In-First-Out).
- Arrays in JavaScript can work both as a queue and as a stack. They allow you to add/remove elements, both to/from the beginning or the end.
- In computer science, the data structure that allows this, is called deque.

Array Print Using Loop:

For loop:

```
let arr = ["Apple", "Orange", "Pear"];  
for (let i = 0; i < arr.length; i++) {  
  alert( arr[i] );  
}
```

For..of:

```
let fruits = ["Apple", "Orange", "Plum"];  
// iterates over array elements  
for (let fruit of fruits) {  
  alert( fruit );  
}
```

Methods that work with the end of the array:

- **Pop**

```
let fruits = ["Apple", "Orange", "Pear"];  
alert(fruits.pop() ); // remove "Pear" and alert it  
alert( fruits ); // Apple, Orange
```

Both `fruits.pop()` and `fruits.at(-1)` return the last element of the array, but `fruits.pop()` also modifies the array by removing it.

- **Push**

```
let fruits = ["Apple", "Orange"];  
fruits.push("Pear");  
alert( fruits ); // Apple, Orange, Pear
```

The call `fruits.push(...)` is equal to `fruits[fruits.length] = ....`

- **Shift**

```
let fruits = ["Apple", "Orange", "Pear"];  
alert( fruits.shift() ); // remove Apple and alert it  
alert( fruits ); // Orange, Pear
```

- **Unshift**

```
let fruits = ["Orange", "Pear"];  
fruits.unshift('Apple');  
alert( fruits ); // Apple, Orange, Pear
```

- **Slice**

It returns a new array copying to it all items from index start to end (not including end). Both start and end can be negative, in that case position from array end is assumed.

It's similar to a string method **str.slice**, but instead of substrings it makes subarrays.

For instance:

```
let arr = ["t", "e", "s", "t"];  
alert( arr.slice(1, 3) ); // e,s (copy from 1 to 3)  
alert( arr.slice(-2) ); // s,t (copy from -2 till the end)
```

• Concat

```
let arr = [1, 2];  
// create an array from: arr and [3,4]  
alert( arr.concat([3, 4]) ); // 1,2,3,4  
// create an array from: arr and [3,4] and [5,6]  
alert( arr.concat([3, 4], [5, 6]) ); // 1,2,3,4,5,6  
// create an array from: arr and [3,4], then add values 5 and 6  
alert( arr.concat([3, 4], 5, 6) ); // 1,2,3,4,5,6
```

Searching in array

- IndexOf/lastIndexOf and includes
- The methods `arr.indexOf` and `arr.includes` have the similar syntax and do essentially the same as their string counterparts, but operate on items instead of characters:
- `arr.indexOf(item, from)` – looks for item starting from index from, and returns the index where it was found, otherwise -1.
- `arr.includes(item, from)` – looks for item starting from index from, returns true if found.
- Usually these methods are used with only one argument: the item to search. By default, the search is from the beginning.

CREATIVE DESIGN & MULTIMEDIA INSTITUTE

```
let arr = [1, 0, false];  
alert( arr.indexOf(0) ); // 1  
alert( arr.indexOf(false) ); // 2  
alert( arr.indexOf(null) ); // -1  
alert( arr.includes(1) ); // true
```

If we want to check if item exists in the array, and don't need the index, then `arr.includes` is preferred.

The method `arr.lastIndexOf` is the same as `indexOf`, but looks for from right to left.

```
let fruits = ['Apple', 'Orange', 'Apple']  
alert( fruits.indexOf('Apple') ); // 0 (first Apple)  
alert( fruits.lastIndexOf('Apple') ); // 2 (last Apple)
```

find and findIndex/findLastIndex

```
let result = arr.find(function(item, index, array) {  
  // if true is returned, item is returned and iteration is stopped  
  // for falsy scenario returns undefined  
});
```

The function is called for elements of the array, one after another:

- **item** is the element.
- **index** is its index.
- **array** is the array itself.

```
let users = [  
  {id: 1, name: "John"},  
  {id: 2, name: "Pete"},  
  {id: 3, name: "Mary"}  
];  
let user = users.find(item => item.id == 1);  
alert(user.name); // John
```

- In real life arrays of objects is a common thing, so the find method is very useful.
- Note that in the example we provide to find the function item => item.id == 1 with one argument. That's typical, other arguments of this function are rarely used.
- The arr.findIndex method has the same syntax, but returns the index where the element was found instead of the element itself. The value of -1 is returned if nothing is found.
- The arr.findLastIndex method is like findIndex, but searches from right to left, similar to lastIndexOf.

```
let users = [  
  {id: 1, name: "John"},  
  {id: 2, name: "Pete"},  
  {id: 3, name: "Mary"},  
  {id: 4, name: "John"}  
];  
// Find the index of the first John  
alert(users.findIndex(user => user.name == 'John')); // 0  
// Find the index of the last John  
alert(users.findLastIndex(user => user.name == 'John')); // 3
```


● Filter

- The find method looks for a single (first) element that makes the function return true.
- If there may be many, we can use `arr.filter(fn)`.
- The syntax is similar to find, but filter returns an array of all matching elements:

```
let results = arr.filter(function(item, index, array) {  
  // if true item is pushed to results and the iteration continues  
  // returns empty array if nothing found  
});
```

Example:

```
let users = [  
  {id: 1, name: "John"},  
  {id: 2, name: "Pete"},  
  {id: 3, name: "Mary"}  
];  
  
// returns array of the first two users  
let someUsers = users.filter(item => item.id < 3);  
  
alert(someUsers.length); // 2
```

Transform an array

- **Map**

The `arr.map` method is one of the most useful and often used.

It calls the function for each element of the array and returns the array of results.

Syntax:

```
let result = arr.map(function (item, index, array) {  
  // returns the new value instead of item  
});
```

Example:

```
let lengths = ["Bilbo", "Gandalf", "Nazgul"].map(item  
=> item.length);  
alert(lengths); // 5,7,6
```

- **sort(fn)**

The call to `arr.sort()` sorts the array in place, changing its element order.

It also returns the sorted array, but the returned value is usually ignored, as `arr` itself is modified.

```
let arr = [ 1, 2, 15 ];  
// the method reorders the content of arr  
arr.sort();  
alert( arr ); // 1, 15, 2
```

- **reverse**

The method `arr.reverse` reverses the order of elements in `arr`.

```
let arr = [1, 2, 3, 4, 5];  
arr.reverse();  
alert( arr ); // 5,4,3,2,1
```

- **split and join**

Here's the situation from real life. We are writing a messaging app, and the person enters the comma-delimited list of receivers: John, Pete, Mary. But for us an array of names would be much more comfortable than a single string. How to get it?

The `str.split(delim)` method does exactly that. It splits the string into an array by the given delimiter `delim`.

In the example below, we split by a comma followed by space:

```
let names = 'Bilbo, Gandalf, Nazgul';  
let arr = names.split(', ');  
for (let name of arr) {  
  alert( `A message to ${name}.` ); // A message to Bilbo (and  
  other names)  
}
```

The `split` method has an optional second numeric argument – a limit on the array length. If it is provided, then the extra elements are ignored. In practice it is rarely used though:

```
let arr = 'Bilbo, Gandalf, Nazgul, Saruman'.split(', ', 2);  
alert(arr); // Bilbo, Gandalf
```

Destructuring

The two most used data structures in JavaScript are object and Array.

- Objects allow us to create a single entity that stores data items by key.
- Arrays allow us to gather data items into an ordered list.

- **Array Destructuring:**

```
// we have an array with the name and surname
let arr = ["John", "Smith"]
// destructuring assignment
// sets firstName = arr[0]
// and surname = arr[1]
let [firstName, surname] = arr;
```

- **Object Destructuring**

```
let options = {
  title: "Menu",
  width: 100,
  height: 200
};
let {title, width, height} = options;
alert(title); // Menu
alert(width); // 100
alert(height); // 200
alert(firstName); // John
alert(surname); // Smith
```

JSON.stringify:

The JSON (JavaScript Object Notation) is a general format to represent values and objects. It is described as in RFC 4627 standard. Initially it was made for JavaScript, but many other languages have libraries to handle it as well. So it's easy to use JSON for data exchange when the client uses JavaScript and the server is written on Ruby/PHP/Java/Whatever.

JavaScript provides methods:

- JSON.stringify to convert objects into JSON.
- JSON.parse to convert JSON back into an object.

```
let student = {
  name: 'John',
  age: 30,
  isAdmin: false,
  courses: ['html', 'css', 'js'],
  spouse: null
};
let json = JSON.stringify(student);
alert(typeof json); // we've got a string!
alert(json);
/* JSON-encoded object:
{
  "name": "John",
  "age": 30,
  "isAdmin": false,
  "courses": ["html", "css", "js"],
  "spouse": null
}
*/
```

Spread Operator:

The JavaScript spread operator (...) allows us to quickly copy all or part of an existing array or object into another array or object.

Array Spread:

```
let arr = [1,2,3];  
let arr2 = [4,5];  
  
arr = [...arr,...arr2];  
console.log(arr); // [ 1, 2, 3, 4, 5 ]
```

Object Spread:

```
const myVehicle = {  
  brand: 'Ford',  
  model: 'Mustang',  
  color: 'red'  
}  
  
const updateMyVehicle = {  
  type: 'car',  
  year: 2021,  
  color: 'yellow'  
}  
  
const myUpdatedVehicle = {...myVehicle, ...updateMyVehicle}  
  
//Check the result object in the console:  
console.log(myUpdatedVehicle);
```

Rest Operator:

Assign the first and second items from numbers to variables and put the rest in an array:

```
const numbers = [1, 2, 3, 4, 5, 6];  
const [one, two, ...rest] = numbers;  
document.write("<p>" + one + "</p>");  
document.write("<p>" + two + "</p>");  
document.write("<p>" + rest + "</p>");
```

Variable scope

- JavaScript is a very function-oriented language. It gives us a lot of freedom. A function can be created at any moment, passed as an argument to another function, and then called from a totally different place of code later.
- We already know that a function can access variables outside of it ("outer" variables).
- But what happens if outer variables change since a function is created? Will the function get newer values or the old ones?
- And what if a function is passed along as an argument and called from another place of code, will it get access to outer variables at the new place?

There Three types of Variable Declaration:

- **var**
- **Let**
- **Const**

Scheduling: `setTimeout` and `setInterval`

There are two methods for it:

- **`setTimeout`** allows us to run a function once after the interval of time.
- **`setInterval`** allows us to run a function repeatedly, starting after the interval of time, then repeating continuously at that interval.

`setTimeout`

Syntax:

```
let timerId = setTimeout(func|code, [delay], [arg1], [arg2], ...)
```

Parameters:

`func|code`

Function or a string of code to execute. Usually, that's a function. For historical reasons, a string of code can be passed, but that's not recommended.

`delay`

The delay before run, in milliseconds (1000 ms = 1 second), by default 0.

`arg1, arg2...`

Arguments for the function

For instance, this code calls `sayHi()` after one second:

```
function sayHi() {  
  alert('Hello');  
}  
  
setTimeout(sayHi, 1000);
```


setInterval

```
let timerId = setInterval(func|code, [delay], [arg1], [arg2], ...)
```

All arguments have the same meaning. But unlike `setTimeout` it runs the function not only once, but regularly after the given interval of time.

To stop further calls, we should call `clearInterval(timerId)`.

```
// repeat with the interval of 2 seconds  
let timerId = setInterval(() => alert('tick'), 2000);  
  
// after 5 seconds stop  
setTimeout(() => { clearInterval(timerId); alert('stop'); }, 5000);
```

Promise:

```
const myPromise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    if(true){  
      resolve("data");  
    }else  
    {  
      reject("error")  
    }, 3000);  
  });  
  myPromise.then((a) => {  
    console.log(a)  
  }).catch((b) => {  
    console.log(b)  
  })  
})
```

Async/Await:

```
const myPromise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    if(true){  
      resolve("data");  
    }else  
    {  
      reject("error")  
    }, 3000);  
  });
```

```
async function getData() {  
  try{  
    const data = await myPromise;  
    console.log(data)  
  }  
  Catch(err){  
    console.log(err)  
  }  
}
```

```
getdata()
```